MySQL

Definition

MySQL is an **open-source relational database management system (RDBMS)** that uses **Structured Query Language (SQL)** to store, manage, and retrieve data. It organizes data into **tables** (rows & columns) and allows users to perform operations like inserting, updating, deleting, and querying data.

- **Use Case:** Websites, applications, data analytics, and enterprise systems.

SELECT Statement

Definition

The **SELECT** statement is used to **retrieve data** from one or more tables in a database.

Syntax

SELECT column1, column2, ...

FROM table_name;

- SELECT → Specifies which columns to retrieve.
- FROM → Specifies the table to retrieve data from.

Athar Ahmed

Example

Suppose you have a table named **employees**:

emp_id	name	department	salary
1	John	HR	4000
2	Maria	IT	6000
3	David	Finance	5000

Query:

SELECT name, salary

FROM employees;

Output:

name	salary
John	4000
Maria	6000
David	5000

WHERE Clause

Definition

The **WHERE** clause is used to **filter records** that meet a specific condition.

Syntax

SELECT column1, column2, ...

FROM table_name

WHERE condition;

- condition → Can include operators like =, <, >, BETWEEN, LIKE, IN, etc.

Example

Get employees with salary **greater than 4500**:

SELECT name, salary

FROM employees

WHERE salary > 4500;

Output:

name salary

Maria 6000

David 5000

Athar Ahmed

Aliases

Aliases are **temporary names** given to **columns or tables** to make results easier to read or queries easier to write.
They **do not change the original name** of the table or column.

---

a) Column Alias

Definition

A **column alias** renames a column in the output.

Syntax

SELECT column_name AS alias_name

FROM table_name;

- AS is optional.

Example

Rename salary as Monthly_Salary:

SELECT name AS Employee_Name,

salary AS Monthly_Salary

FROM employees;

Output:

Employee_Name	Monthly_Salary
John	4000
Maria	6000
David	5000

Athar Ahmed

---

b) Table Alias

Definition

A **table alias** gives a temporary name to a table, useful when working with **long table names** or **joins**.

Syntax

SELECT t.column1, t.column2

FROM table_name AS t;

Example

```
SELECT e.name, e.salary
FROM employees AS e
WHERE e.department = 'IT';
```

This query gives the same result but uses e instead of writing employees repeatedly.

Combined Example

Retrieve employee names and their salaries (renamed as Pay), from table employees (aliased as e), where salary is greater than 4500:

```
SELECT e.name AS Employee,
    e.salary AS Pay
FROM employees AS e
WHERE e.salary > 4500;
```

Output:

Employee	Pay
Maria	6000
David	5000

Athar Ahmed

Key Takeaways

CONCEPT	PURPOSE	KEYWORD
SELECT	Retrieve data from a table	SELECT
FROM	Specify the table to query	FROM
WHERE	Filter rows based on a condition	WHERE
ALIAS	Temporary name for column or table	AS

ORDER BY in MySQL

Definition

The **ORDER BY** clause is used to **sort the result set** of a query in **ascending (default)** or **descending** order based on one or more columns.

Syntax

SELECT column1, column2, ...

FROM table_name

ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];

- **ORDER BY** → Specifies which column(s) to sort by.
- **ASC** → Ascending order (default, from smallest to largest).
- **DESC** → Descending order (from largest to smallest).
- You can sort by **multiple columns** (first column decides primary sort order).

Sample Table: employees

emp_id	name	department	salary
1	John	HR	4000
2	Maria	IT	6000
3	David	Finance	5000
4	Anna	IT	7000

Athar Ahmed

Sort in Ascending Order (Default)

List employees by **salary** from lowest to highest:

SELECT name, salary

FROM employees

ORDER BY salary;

Output:

name	salary
John	4000
David	5000
Maria	6000
Anna	7000

(ASC is optional because it's default)

Sort in Descending Order

List employees by **salary** from highest to lowest:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Output:

name	salary
Anna	7000
Maria	6000
David	5000
John	4000

Athar Ahmed

Sort by Multiple Columns

Sort employees **by department (A→Z)**, and **within each department by salary (high→low)**:

```
SELECT name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Output:

name	department	salary
David	Finance	5000
John	HR	4000
Anna	IT	7000
Maria	IT	6000

Sort by Column Alias

You can sort using an **alias** defined in the SELECT statement.

Example: Rename salary as Pay and sort by Pay descending:

```
SELECT name, salary AS Pay
FROM employees
ORDER BY Pay DESC;
```

Athar Ahmed

Sort by Expression

You can sort using **calculated expressions**.

Example: Sort by salary **after adding a 10% bonus**:

```
SELECT name, salary, (salary * 1.1) AS New_Salary
FROM employees
ORDER BY New_Salary DESC;
```

Key Points:

Feature	Example	Meaning
Ascending order	ORDER BY salary ASC	Lowest to highest (default)

Descending order	ORDER BY salary DESC	Highest to lowest
Multiple columns	ORDER BY department, salary	Sort by department, then salary
Alias in ORDER BY	ORDER BY Pay DESC	Sort by column alias
Expression in ORDER BY	ORDER BY salary*1.1	Sort by calculated value

---

Combined Example

Show employees in **IT department**, sorted by **salary descending**, and rename columns:

```
SELECT name AS Employee,
    department AS Dept,
    salary AS Monthly_Salary
FROM employees
WHERE department = 'IT'
ORDER BY Monthly_Salary DESC;
```

Output:

Employee	Dept	Monthly_Salary
Anna	IT	7000
Maria	IT	6000

Athar Ahmed

INSERT – Add New Records

Definition

The **INSERT** statement is used to **add new rows** (records) into a table.

Syntax

```
-- Insert into specific columns
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);


-- Insert into all columns
```

```
INSERT INTO table_name
VALUES (value1, value2, ...);
```

Example Table: employees

emp_id	name	department	salary
1	John	HR	4000
2	Maria	IT	6000

Example 1: Insert into specific columns

Add a new employee:

```
INSERT INTO employees (emp_id, name, department, salary)
VALUES (3, 'David', 'Finance', 5000);
```

Resulting Table:

emp_id	name	department	salary
1	John	HR	4000
2	Maria	IT	6000
3	David	Finance	5000

Athar Ahmed

Example 2: Insert multiple rows

```
INSERT INTO employees (emp_id, name, department, salary)
VALUES
(4, 'Anna', 'IT', 7000),
(5, 'Steve', 'HR', 4500);
```

UPDATE – Modify Existing Records

Definition

The **UPDATE** statement is used to **change existing data** in one or more rows.

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

- **Always use WHERE** to avoid updating **all rows**.

Example 1: Update a single row

Increase salary of **John** to 4500:

```
UPDATE employees
SET salary = 4500
WHERE name = 'John';
```

Example 2: Update multiple columns

Change **Steve's** department to IT and salary to 4800:

```
UPDATE employees
SET department = 'IT',
    salary = 4800
WHERE name = 'Steve';
```

Example 3: Update multiple rows

Increase salary of all IT employees by 10%:

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

Athar Ahmed

DELETE – Remove Records

Definition

The **DELETE** statement is used to **remove rows** from a table.

Syntax

```
DELETE FROM table_name
WHERE condition;
```

- **Always use WHERE** to avoid deleting **all rows**.

Example 1: Delete a single row

Remove employee **David**:

```
DELETE FROM employees
WHERE name = 'David';
```

Example 2: Delete multiple rows

Remove all employees in **HR** department:

```
DELETE FROM employees
WHERE department = 'HR';
```

Important Notes

Command	Purpose	Key Caution
INSERT	Add new rows to a table	Ensure values match column data types
UPDATE	Change existing row values	Use WHERE to avoid updating all rows
DELETE	Remove rows from a table	Use WHERE to avoid deleting all rows

---

Full Workflow Example

Suppose we start with:

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10,2)
);
```

Athar Ahmed

Insert Data

```
INSERT INTO employees (emp_id, name, department, salary)
VALUES (1, 'John', 'HR', 4000),
       (2, 'Maria', 'IT', 6000),
       (3, 'David', 'Finance', 5000);
```

Update Data

```
UPDATE employees
SET salary = 5200
WHERE name = 'David';
```

Delete Data

```
DELETE FROM employees
WHERE emp_id = 1;
```

Quick Summary

Command	Action	Example
INSERT	Add new data	INSERT INTO employees VALUES (4, 'Anna', 'IT', 7000);
UPDATE	Modify data	UPDATE employees SET salary = 4500 WHERE emp_id = 2;
DELETE	Remove data	DELETE FROM employees WHERE department = 'HR';

Athar Ahmed